

REPORT
OF
**Virtual Internship in
Artificial Intelligence**

Submitted

To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE



By

Collage Roll Number. 2410031459

Student Internship ID: CA/DF1/206810

Name of Student: NAVEEN KUMAR

Batch: 2024-28

CANDIDATE’S DECLARATION

I, NAVEEN KUMAR, do hereby declare that the internship report titled “AI-Based Language Translation, Music Generation and Object Detection & Tracking” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. The work presented in this report is based on the internship tasks assigned to me and the technical workflow described in the project brief. References have been acknowledged wherever external concepts, frameworks, APIs or research methods are discussed. I affirm that this report has not been submitted to any other institute for the same degree or diploma requirement and I shall be responsible for the accuracy and originality of the submitted work.

Student Name: NAVEEN KUMAR

Collage Roll No.2410031459

Student Internship ID: CA/DF1/206810

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for the guidance, constructive criticism and friendly advice that helped me understand practical applications of artificial intelligence. I am sincerely grateful to the internship organization, mentors and academic faculty for providing an environment in which I could explore language translation, generative music and computer vision workflows. I also express my gratitude to my parents for their blessings and encouragement.

Student Name: NAVEEN KUMAR

Collage Roll No.2410031459

Student Internship ID: CA/DF1/206810

CONTENTS

1. Candidate's Declaration	i
2. Acknowledgment	ii
3. Internship Completion Certificate	iii
List of Figures	iv
List of Tables	v
4. Project Description	1
4.1 Introduction	1
4.2 Organization Profile	2
4.3 Problem Statement	2
4.4 Project Objectives	3
4.5 Scope of the Project	3
4.6 Technologies and Tools Used	4
4.7 System Architecture	5
4.8 Methodology	6
4.9 Expected Outcomes	11
4.10 Certificates of Completion and Other Proof	12

LIST OF FIGURES

Figure 4.1 Integrated AI Internship Project Architecture

Figure 4.2 Language Translation Workflow

Figure 4.3 AI Music Generation Pipeline

Figure 4.4 Object Detection and Tracking Pipeline

LIST OF TABLES

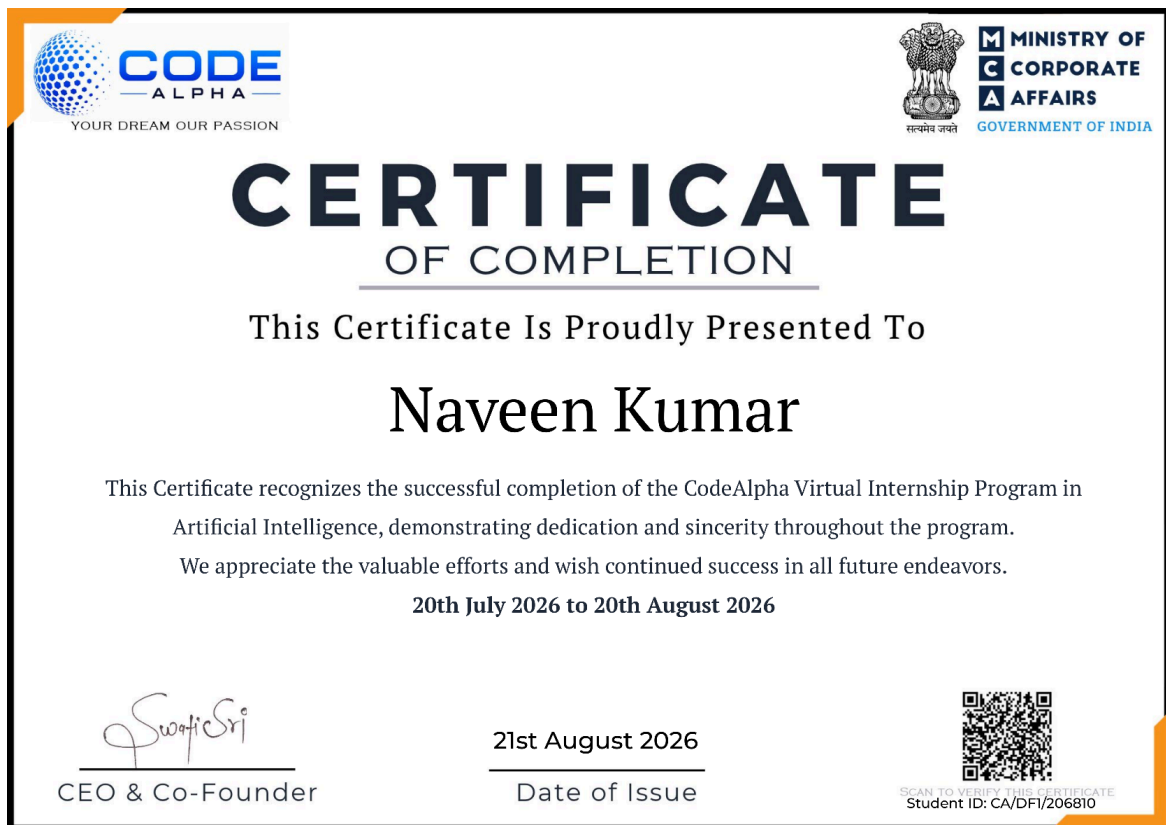
Table 4.1 Technology Stack

Table 4.2 Task-wise Objective Matrix

Table 4.3 Expected Outcomes and Validation

INTERNSHIP COMPLETION CERTIFICATE

CodeAlpha Virtual Internship Program - Artificial Intelligence



Certificate of Completion issued to NAVEEN KUMAR. The attached certificate states the internship period as 20 July 2026 to 20 August 2026 and the date of issue as 21 August 2026.

CHAPTER 4 PROJECT DESCRIPTION

4.1 Introduction

The internship project brings together three practical artificial intelligence tasks: a language translation tool, an AI-based music generation pipeline, and a real-time object detection and tracking system. The three tasks represent complementary AI capabilities. Translation focuses on natural-language processing and API integration; music generation focuses on sequence modelling and generative learning; and object detection and tracking focuses on computer vision and temporal identity association.

The project is designed as a modular workflow so that each task can be developed, tested and demonstrated independently while following a common pattern of input processing, AI inference, output generation and user-facing presentation. The task descriptions supplied for the internship specify API-based translation, MIDI preprocessing with music21, recurrent models such as LSTM/RNNs, YOLO/Faster R-CNN detection, and SORT/Deep SORT tracking.

A major objective of the report is therefore to document not only the individual tasks but also the engineering decisions needed to turn them into demonstrable applications: input validation, data preprocessing, model selection, inference flow, output formatting, and usability considerations.

4.2 Organization Profile

According to the supplied internship completion certificate, the internship was completed under the CodeAlpha Virtual Internship Program in Artificial Intelligence. The certificate recognizes successful completion of the program and records the internship period as 20th July 2026 to 20th August 2026, with the certificate issued on 21st August 2026. The student name on the certificate is NAVEEN KUMAR and the Student Internship ID is CA/DF1/206810.

Within the internship scope documented for this report, the technical work is centered on practical AI application building rather than a single monolithic model. The three tasks are intentionally diverse and together provide exposure to NLP, generative modelling, deep learning, computer vision, real-time processing and interface design.

4.3 Problem Statement

The project addresses three real-world usability and automation problems. First, users need a simple interface for converting text from one language to another without

manually managing API requests. Second, generating original symbolic music requires a model that can learn patterns in note sequences and produce new sequences in a structured MIDI-compatible form. Third, interpreting live video requires a system that can detect multiple object categories and maintain their identities across consecutive frames.

The resulting system must therefore satisfy the following technical requirements: accept practical user inputs; transform inputs into formats expected by the selected AI component; invoke or execute the AI model reliably; handle errors and edge cases; and present outputs in a clear, usable form.

4.4 Project Objectives

- Build a user-facing language translation interface with source and target language selection.
- Integrate a translation API and display the translated response clearly, with optional copy or text-to-speech support.
- Collect and preprocess MIDI music data into note/chord sequences suitable for neural sequence learning.
- Design an RNN/LSTM-based generative model capable of learning musical sequence patterns and producing new symbolic music.
- Convert generated sequences back into MIDI and provide a path for playback or audio rendering.
- Capture webcam or video-file frames, detect objects using a pre-trained detector, and visualize bounding boxes with class labels.
- Attach stable tracking IDs using SORT or Deep SORT so that detected objects can be followed across frames.
- Present the three tasks as modular AI applications with clear input-output flows and validation criteria.

Task	Primary Objective	AI Area	User-visible Output
Task 1	Translate user-provided text between selected languages.	NLP / API Integration	Translated text; optional copy/TTS.
Task 2	Learn musical patterns and generate new note sequences.	Generative AI / Sequence Learning	Generated MIDI and optional audio playback.
Task 3	Detect objects and maintain identities in video frames.	Computer Vision	Boxes, labels, confidence and track IDs.

Table 4.2 Task-wise Objective Matrix

4.5 Scope of the Project

The scope covers the complete conceptual and implementation workflow required to demonstrate the three assigned AI tasks. For translation, the scope includes interface controls, request formation, API response handling and output presentation. For music generation, the scope includes MIDI collection, parsing, sequence construction, recurrent neural-network training, sequence generation and MIDI export. For object detection and tracking, the scope includes frame capture, object detection, tracking association, annotation and real-time display.

The scope does not claim a specific production deployment, a measured benchmark score, or a fixed cloud architecture because those details were not specified in the supplied task brief. Such values should be added only when supported by experiment logs or deployment evidence.

4.6 Technologies and Tools Used

Technology / Tool	Role	Use in Project
Python	Primary development language for all three tasks.	Rapid prototyping, AI/ML libraries and application logic.
Translation API	Google Translate API or Microsoft Translator.	Remote translation inference and multilingual text conversion.
music21	MIDI/music representation and preprocessing.	Parsing scores, extracting notes/chords and structuring sequences.
PyTorch / TensorFlow	Deep learning framework.	Implementing and training RNN/LSTM models.
OpenCV	Computer vision and video I/O.	Webcam/video capture, frame processing and drawing.
YOLO / Faster R-CNN	Pre-trained object detector.	Frame-level object localization and classification.
SORT / Deep SORT	Multi-object tracking method.	Association of detections over time and track IDs.
MIDI utilities	MIDI file creation / playback toolchain.	Saving generated note sequences and enabling playback.
Git / Version Control	Source-code and experiment management.	Tracking changes and organizing project files.

Table 4.1 Technology Stack

4.7 System Architecture

The integrated architecture follows a modular pipeline. User or media input enters an application controller, which routes the request to one of the three task modules. Each

module applies its own preprocessing and AI technique, then returns a structured result to the presentation layer. The design reduces coupling: translation can operate independently of music generation and computer vision, while the common interface pattern keeps the internship work cohesive.

Integrated AI Internship Project Architecture

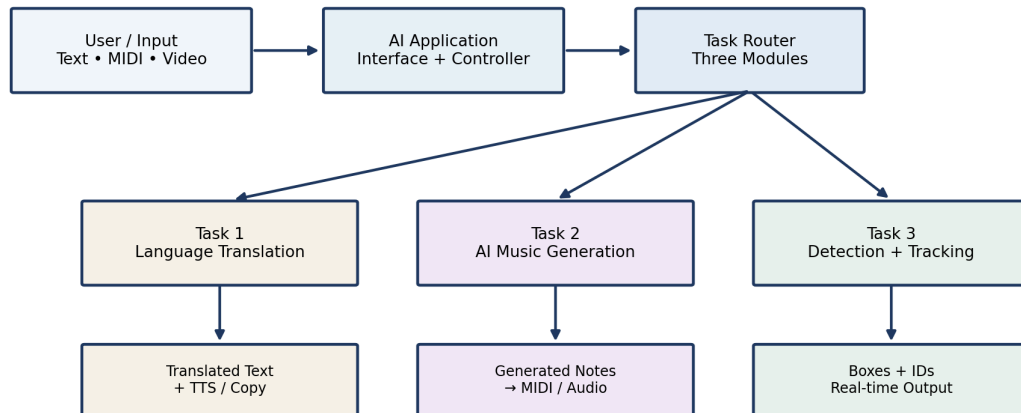


Figure 4.1 Integrated AI Internship Project Architecture

The architecture also supports iterative development. A task can first be demonstrated with a small input, then extended with stronger models, richer datasets, improved user interface elements and more robust error handling. This makes the overall project suitable for an internship context where the emphasis is on understanding and demonstrating the full AI application lifecycle.

4.8 Methodology

4.8.1 Task 1 - Language Translation Tool

The translation module is designed as a lightweight application in which the user enters source text, chooses a source language and a target language, and triggers translation. The application prepares an authenticated request to the selected translation service, sends the text, parses the response and renders the translated result in a dedicated output area.

Input validation should be performed before an API call. Empty text can be rejected; unsupported language selections can be blocked; unusually long requests can be handled with a clear message; and network/API errors should be surfaced without

exposing credentials. API keys or service credentials should be kept outside the source code, for example in environment variables or a secure secret store.

Language Translation Workflow

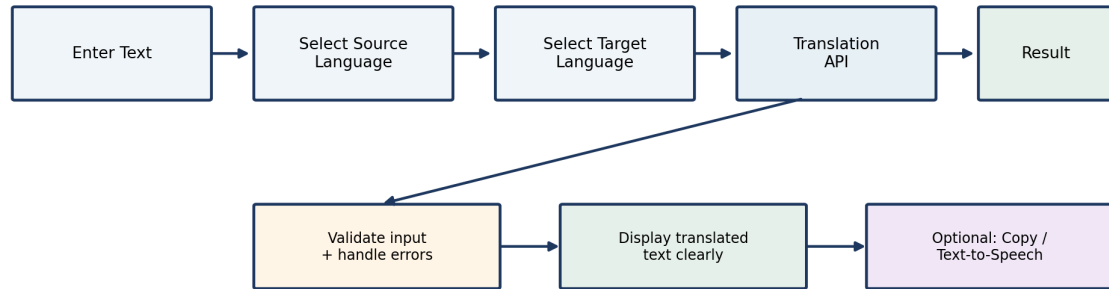


Figure 4.2 Language Translation Workflow

Translation implementation sequence

1. Create the interface fields for input text, source language, target language and a Translate action.
2. Validate the user input and obtain the required API credential from a secure configuration.
3. Format the API request using the provider-specific endpoint and language codes.
4. Send the request and parse the returned translation payload.
5. Display the translated text in an easily readable output panel.
6. Add optional usability features such as a copy-to-clipboard action or text-to-speech playback.
7. Handle API failures, rate limits, invalid parameters and empty input with user-friendly messages.

Illustrative pseudocode: `translate(text, source, target) → validate → build request → call API → parse translation → display result`. The exact request body and authentication mechanism depend on the provider selected for implementation.

4.8.2 Task 2 - Music Generation with AI

The music-generation task treats music as a sequence-learning problem. MIDI files from suitable public-domain or licensed datasets can be parsed with music21. Notes,

rests, durations, velocities and/or chord information can be transformed into a token or numerical representation. Fixed-length windows of these representations then become model inputs, with the next token or sequence position used as the learning target.

An RNN or LSTM model is appropriate because the generated music is sequential: each musical event is influenced by preceding events. During training, the model learns the distribution of note transitions and longer-range patterns present in the dataset. During generation, a seed sequence is supplied and the model predicts a next-event distribution repeatedly until a desired sequence length is reached.

AI Music Generation Pipeline

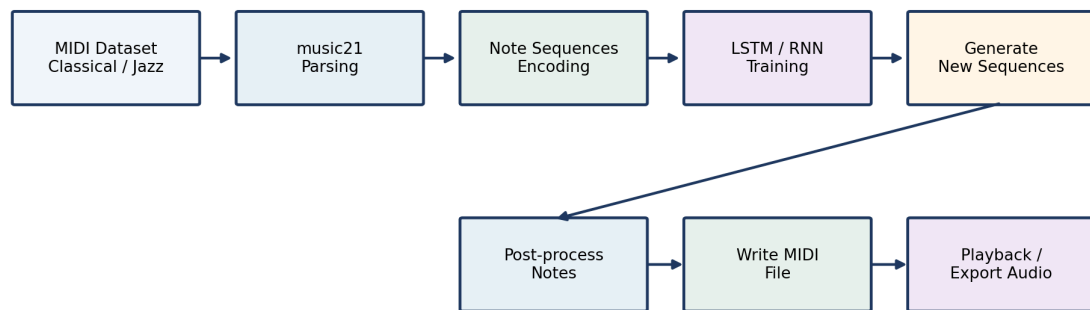


Figure 4.3 AI Music Generation Pipeline

Music preprocessing and generation sequence

8. Collect a consistent set of MIDI files representing the intended style, for example classical or jazz.
9. Parse each MIDI file with music21 and extract musical events into a normalized representation.
10. Remove or standardize problematic events and create a vocabulary of tokens or encoded values.
11. Build overlapping input sequences and targets for supervised sequence prediction.
12. Create an RNN/LSTM model with an embedding or numeric input layer, recurrent layers and an output layer over the event vocabulary.
13. Train the model on the prepared sequences and monitor training behaviour for instability or overfitting.

14. Seed the trained model with a short sequence, sample predicted events and iterate until the target length is reached.
15. Reconstruct the predicted events into a music21 stream and write the result to a MIDI file.
16. Use a MIDI player or audio renderer to listen to the generated composition.

Important preprocessing considerations include keeping a consistent encoding scheme, handling tempo and instrument metadata appropriately, and ensuring generated sequences end on valid musical events. The task brief does not provide a specific dataset size, model depth or training duration, so these are treated as configurable experimental choices rather than reported results.

4.8.3 Task 3 - Object Detection and Tracking

The computer-vision module receives frames from a webcam or video file through OpenCV. Each frame is passed to a pre-trained object-detection model such as YOLO or Faster R-CNN. The detector returns candidate objects with class labels, confidence scores and bounding boxes. Those detections are then passed to a tracking layer based on SORT or Deep SORT, which associates observations from one frame with observations in the next frame and assigns persistent track IDs.

Object Detection and Tracking Pipeline

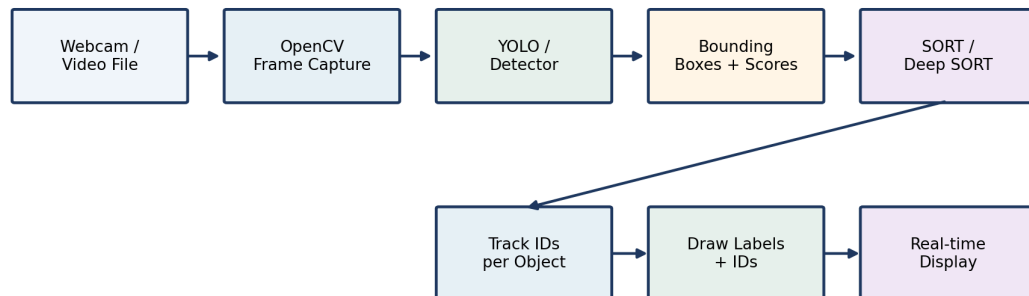


Figure 4.4 Object Detection and Tracking Pipeline

Detection and tracking sequence

17. Initialize the webcam or open a video file and read frames using OpenCV.

18. Run the selected detector on each frame to obtain bounding boxes, classes and confidence scores.
19. Filter detections using a confidence threshold suitable for the demonstration environment.
20. Pass the filtered detections to SORT or Deep SORT for temporal association.
21. Draw the bounding box, class label, confidence value and tracking ID for each active track.
22. Display the annotated frame in real time and release video resources on exit.

Tracking adds temporal context to frame-by-frame detection. A person who moves across the scene can retain the same ID across consecutive frames, subject to the tracker's association logic and the quality of the detector output. In crowded scenes, occlusion and missed detections can cause temporary ID switches; these are normal areas for later optimization.

4.8.4 Common Validation and Error Handling

Although the three tasks use different AI techniques, a common validation strategy improves reliability. The system should validate inputs before processing, isolate secrets and model files from user-visible configuration, record meaningful errors, and provide a visible output state after each action. For the API task, network and authentication errors are important; for music generation, malformed MIDI or unsupported events are key risks; and for tracking, dropped frames, low-confidence detections and occlusions are common sources of output instability.

4.9 Expected Outcomes

The expected outcome is a set of three demonstrable AI applications, each producing an observable result from its input. The report intentionally describes expected outputs rather than inventing numerical performance figures that are not included in the supplied internship brief.

Task	Expected Technical Result	User-visible Result	Validation Approach
Language Translation	Input text is converted from selected source language to selected target language.	Visible translated text; optional copy and TTS support.	Test multiple language pairs, empty input, invalid selections and API failure cases.
AI Music Generation	A trained sequence model generates new musical events from a seed sequence.	MIDI file representing newly generated music; optional audio playback.	Listen for syntactic validity, sequence completion and variety across seeds.

Task	Expected Technical Result	User-visible Result	Validation Approach
Object Detection + Tracking	Video frames are analyzed and objects are associated over time.	Annotated real-time video with labels and persistent track IDs.	Test different scenes, movement, occlusion and changes in object count.

Table 4.3 Expected Outcomes and Validation

Together, these tasks provide practical experience across the AI application lifecycle: data/input handling, preprocessing, model/API execution, output formatting, and user interaction. The project can be extended further by adding richer multilingual UI support, more musical datasets and stronger generative architectures, or higher-accuracy detection and tracking models.

4.9.1 Conclusion

The internship project demonstrates how different AI paradigms can be combined into practical software modules. The translation task emphasizes API integration and NLP workflow design; the music-generation task demonstrates sequential deep learning and symbolic data processing; and the detection-and-tracking task illustrates real-time computer vision. A modular architecture keeps these tasks understandable and testable while providing a clear path toward more advanced implementations.

4.10 Certificates of Completion and Other Communication Proof

The supplied certificate is included in this report as documentary evidence of internship completion. The certificate identifies NAVEEN KUMAR as the recipient, records the CodeAlpha Virtual Internship Program in Artificial Intelligence, gives the internship period as 20th July 2026 to 20th August 2026, and shows the issue date as 21st August 2026.

A copy of the certificate is attached in the front matter of this report.